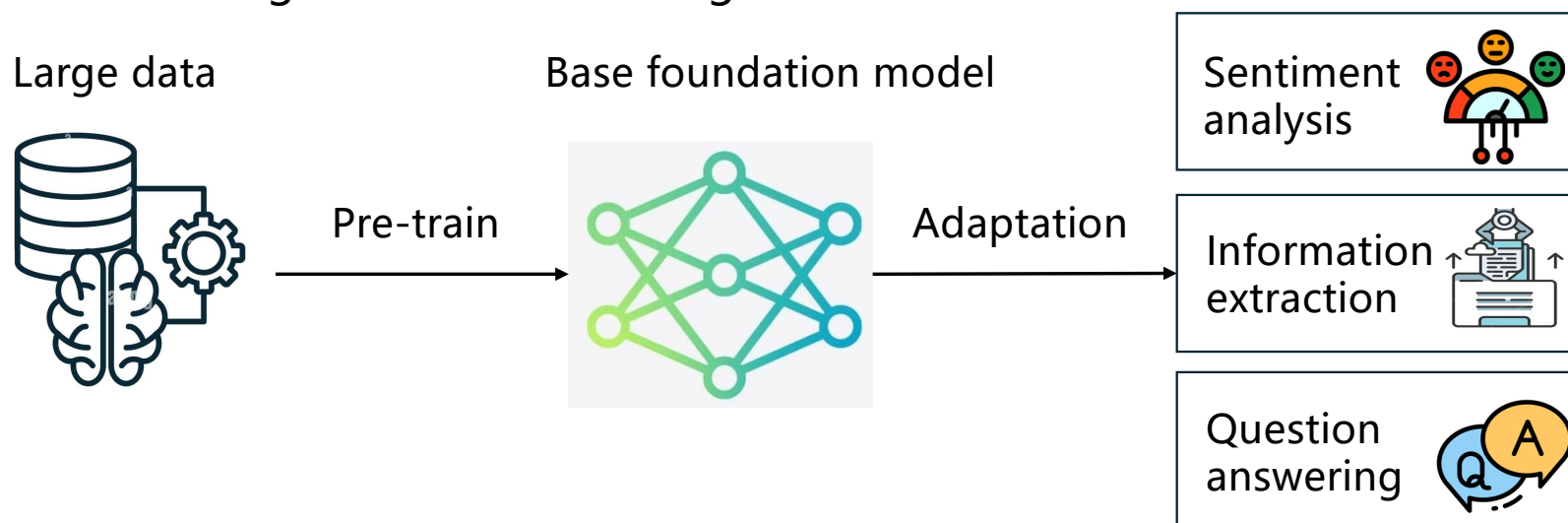


Pretraining, mid-training, post-training

- Pre-training: teach the model the **fundamentals** of language, grammar, reasoning, and a broad range of world knowledge.



- Mid-training: further train a base model on a specific, high-quality domain to **deepen its expertise** without losing its general knowledge.
- Post-training: a.k.a, alignment, transform the base model into an “assistant” that can **follow instructions and align with human values** (helpful, honest, harmless).
 - SFT, RLHF (to be discussed later)

Pretraining, mid-training, post-training

Summary Analogy: Building a Chef

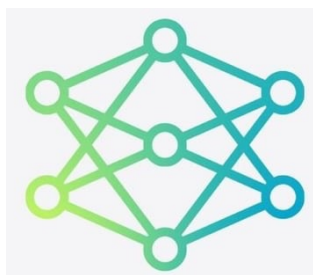
LLM training	Stage	Analogy	Data	Result
	Pre-training	Culinary School (Learning ingredients, techniques, recipes from every cookbook).	Every cookbook and food blog on the internet.	A line cook who knows how to cook <i>anything</i> in theory, but can't take orders.
	Mid-training	Specializing in French Cuisine (Working exclusively in a French kitchen).	Hundreds of French recipes and techniques.	A line cook who is an expert in French cuisine.
	Post-training	Learning the Restaurant's Menu (Practicing the specific 10 dishes the restaurant serves and learning to plate them nicely).	The specific recipes and plating instructions for the restaurant's menu.	A head chef ready to serve customers.

LLM training stages are similar to training a chef.



Pretraining: downstream fine-tuning/prompting

Base foundation model



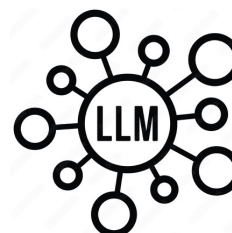
+

Example:
(1+1, 2)

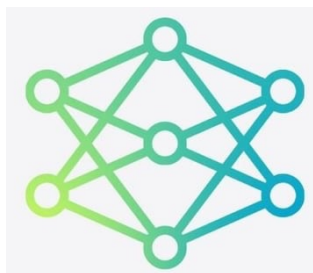
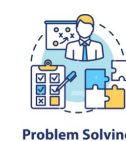


Fine-tune

Fine-tuned model



Problem
solving



+

PROMPT

Example:
 $1+1=2$, so
 $2*(1+1)=?$

Problem
solving

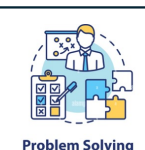


Image Source:

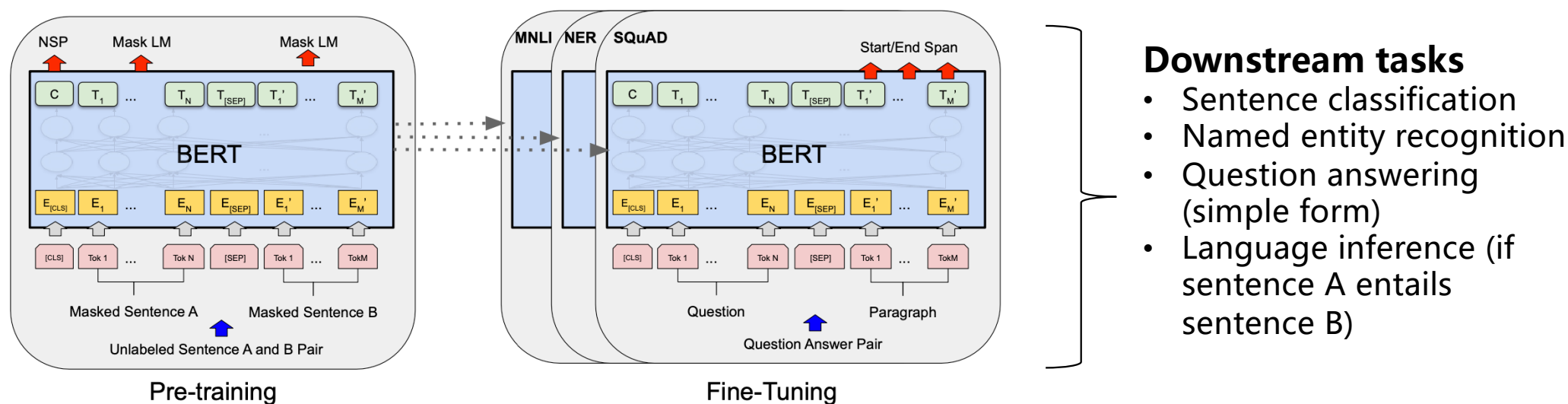
https://www.veryicon.com/icons/education-technology/guangzhou-baiyun-district-ecological/icon_-_task-tracking.html

<https://www.alamy.com/problem-solving-concept-icon-planning-management-way-out-of-difficult-situations-decision-making-idea-thin-line-illustration-vector-isolated-outl-image341371726.html>

https://www.flaticon.com/free-icon/prompt_10817257

Pretraining: why

Transferability Across Tasks: Pretrained models can be adapted to a wide range of tasks through fine-tuning



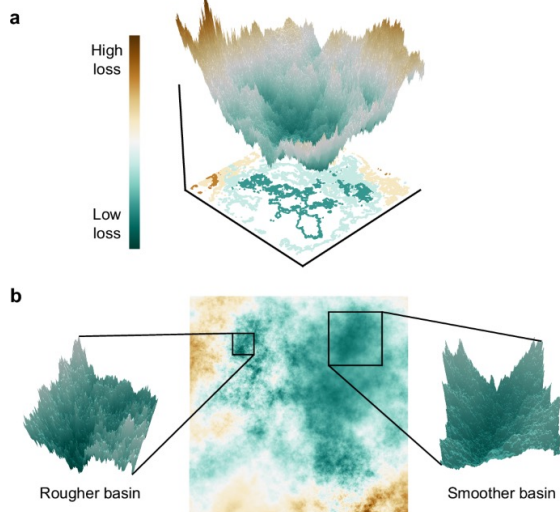
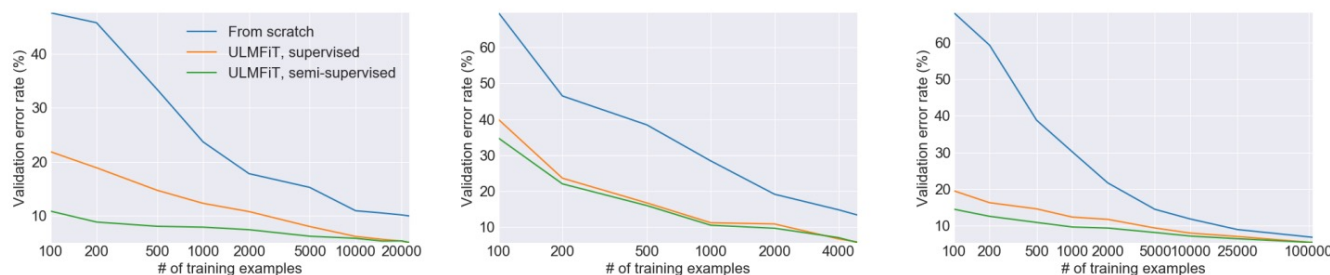
The pre-trained model parameters are used to initialize BERT for **multiple down-stream tasks**.

Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.

Pretraining: why

Improved task performance: better results compared to training from scratch

Validation error rates for supervised and semi-supervised ULMFiT vs. training from scratch with different numbers of training examples on IMDb, TREC-6, and AG.



Why it is easier with pre-training?

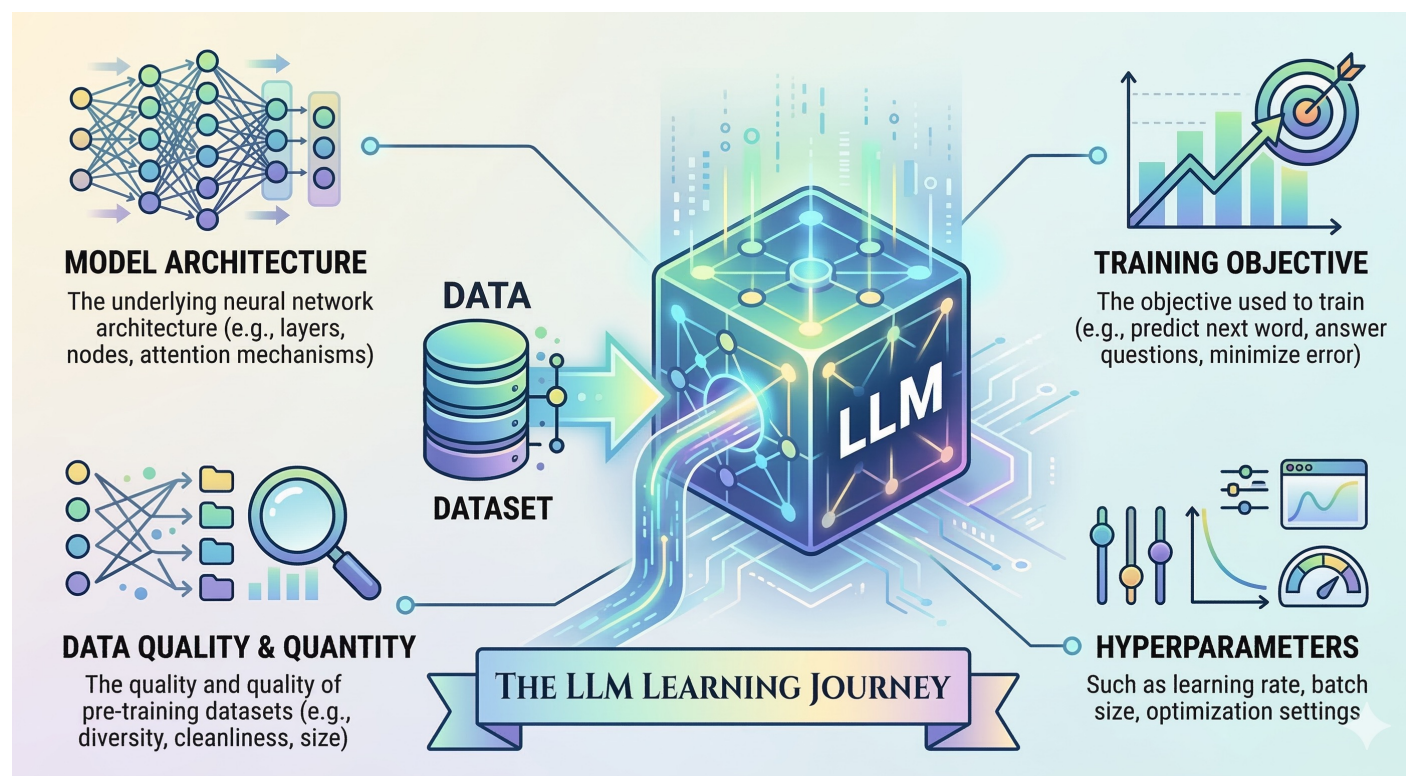
In deep learning, optimizers demonstrate a remarkable ability to dynamically navigate complex loss landscapes, ultimately converging to solutions that generalize well.

Pretraining moves the model parameters to a smoother landscape.

Howard J, Ruder S. Universal language model fine-tuning for text classification. ACL.(Volume 1: Long Papers). 2018: 328-339.
<https://www.nature.com/articles/s41467-025-58532-9>

Major factors affecting pre-training

1. **Model:** the underlying neural network architecture. **important but fixed.*
2. **Training objective:** the objective used to train. **important but fixed.*
3. **Data:** the quantity and quality of pre-training datasets. **very important.*
4. **Hyperparameters:** such as learning rate, batch size. **These are not too critical at the end with large data.*



| Model architecture

- **Typically a Transformer**, though the specifics can differ
 - it learns quite fast from texts, absorbing significant linguistic patterns
 - handle long-range dependencies
 - scaling law: gets stronger scalability as model and **data** sizes increase
- **Size:** Larger models tend to perform better on tasks within a specific model family
- **Model details** can differ.

| What data to use

- Commonly used:
 - Books
 - Wikipedia
 - Common crawl: data from the whole internet
 - Social media data
 - Open source code repositories
- Publicly available data, though still increasing, are effectively used up.
 - Consider private special data (train a robot reasoning model using scene data collected in a factory) and AI-generated data.
- **Quantity:** How much data do I have
- **Quality:** Is the data suitable for training purposes?
- **Coverage:** Does the data adequately cover the domains I care about, and in the right proportions?

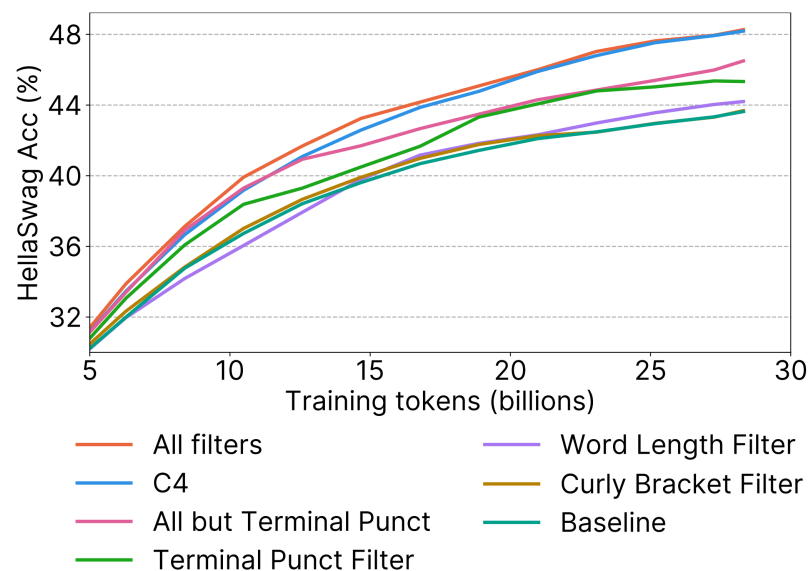
| Data factors: quantity

Data quantity refers to the total amount of data used to train a model, often measured in tokens for large language models

Model	Model size	# of Tokens
Llama 1	7B, 13B, 33B, 65B	1.4 trillion
Llama 2	7B, 13B, 70B	1.8 trillion
Llama 3	8B, 70B, 405B	15 trillion
Deepseek3	671B	15 trillion

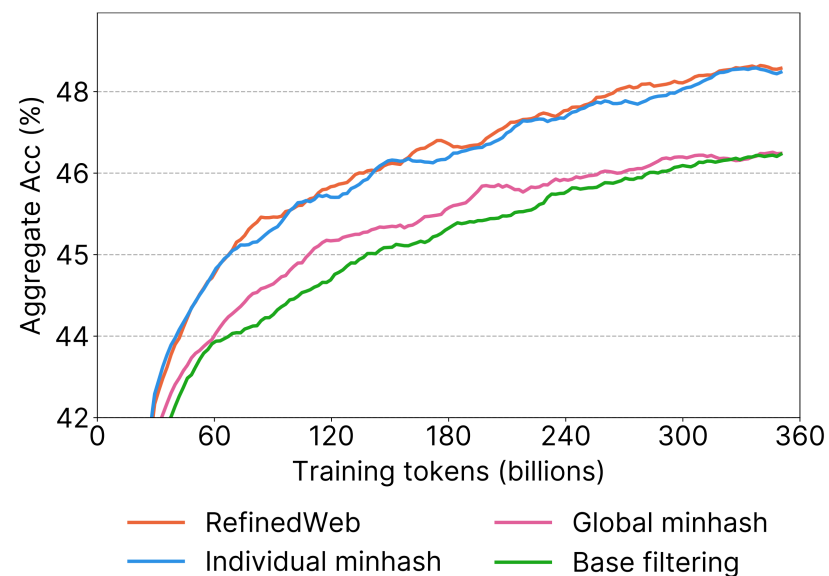
Data factors: quality

Filtering



Filter out unwanted text

Deduplication



Remove duplicate content

Penedo G, Kydliček H, Lozhkov A, et al. The fineweb datasets: Decanting the web for the finest text data at scale. Advances in Neural Information Processing Systems, 2024, 37: 30811-30849.

| Pretraining objective function

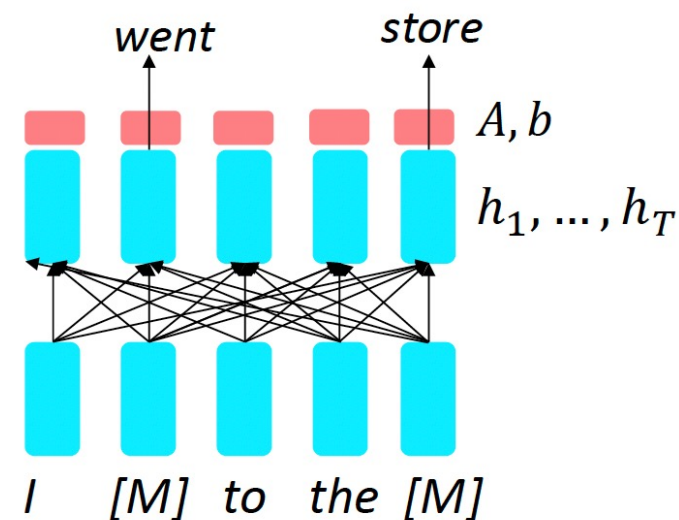
- **Masked language modeling:** used more for fine-tuning.
Example: BERT
- **Auto-regressive language modeling:** used for fine-tuning, prompting.
Example: GPT 2/3/4, Llama1/2

Masked language modeling

- **Idea:** replace some fraction of words in the input with special [MASK] token; predict these words.

- $$h_1, \dots, h_T = \text{Encoder}(w_1, \dots, w_T)$$
$$z_i \sim Ah_i + b$$

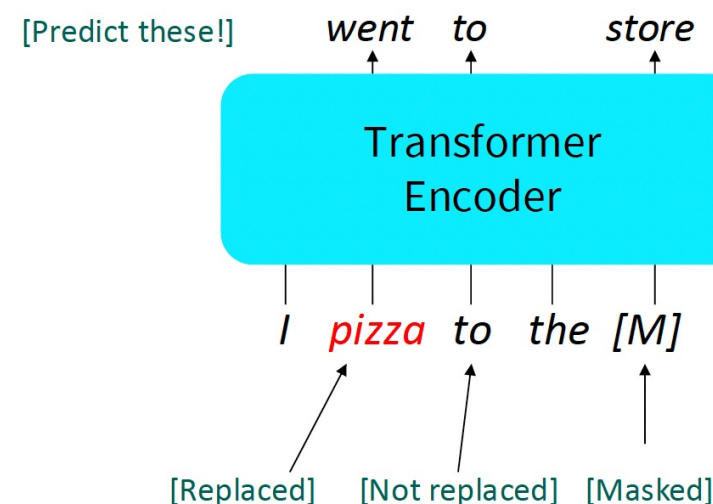
- Only add loss terms from words that are “masked out” . If $\tilde{x}$ is the masked version of x , we are learning $p_\theta(x|\tilde{x})$, called masked LM.



Masked language modeling: BERT

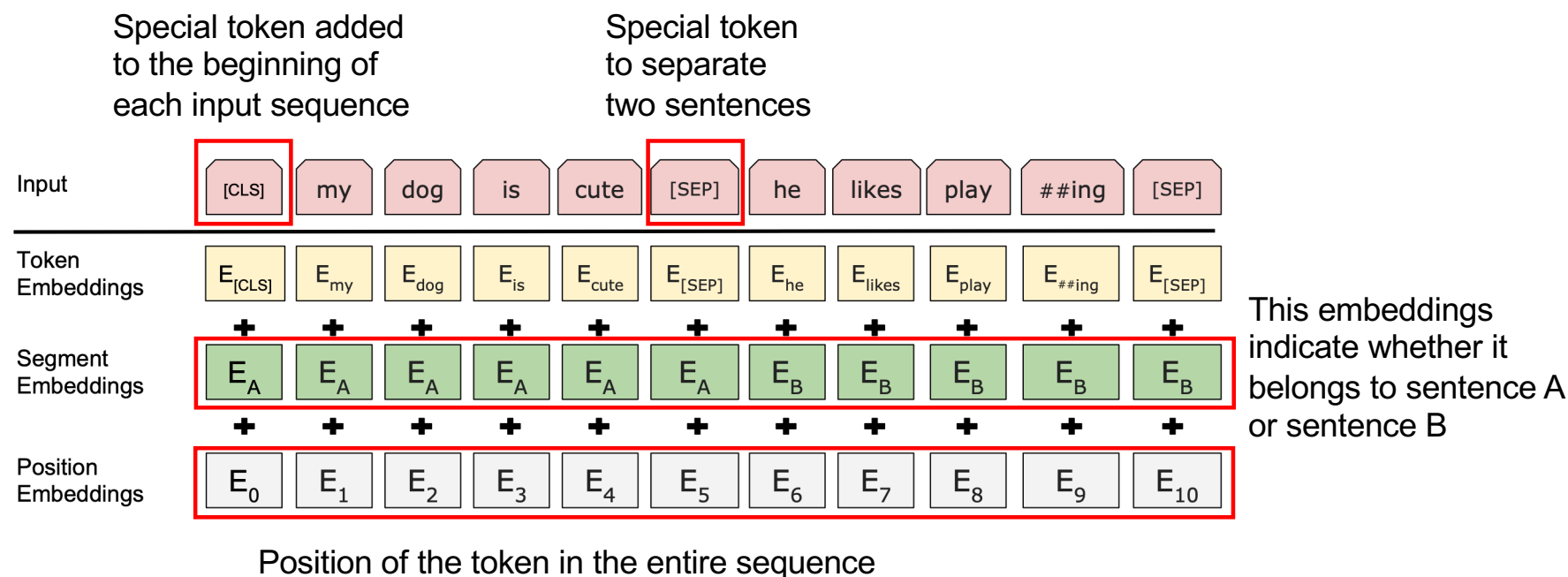
Some details about Masked LM for BERT

- **Predict a random 15% of word tokens.**
 - Replace input word with [Mask] 80% of the time.
 - Replace input word with a random token 10% of the time.
 - Leave input word unchanged 10% of the time (but still predict it)
- **No masks are seen at fine-tuning time**



Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.

Masked language modeling: BERT



The final embedding is the sum of all three

Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.

Masked language modeling: BERT

- Achieved state-of-the-art performance across various tasks after fine-tuning

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

- QQP**: Quora Question Pairs (detect paraphrase questions)
- QNLI**: natural language inference over question answering data
- SST-2**: sentiment analysis
- CoLA**: corpus of linguistic acceptability (detect whether sentences are grammatical.)
- STS-B**: semantic textual similarity
- MRPC**: microsoft paraphrase corpus
- RTE**: a small natural language inference corpus

Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.

Masked language modeling: BERT

- **Two model sizes**

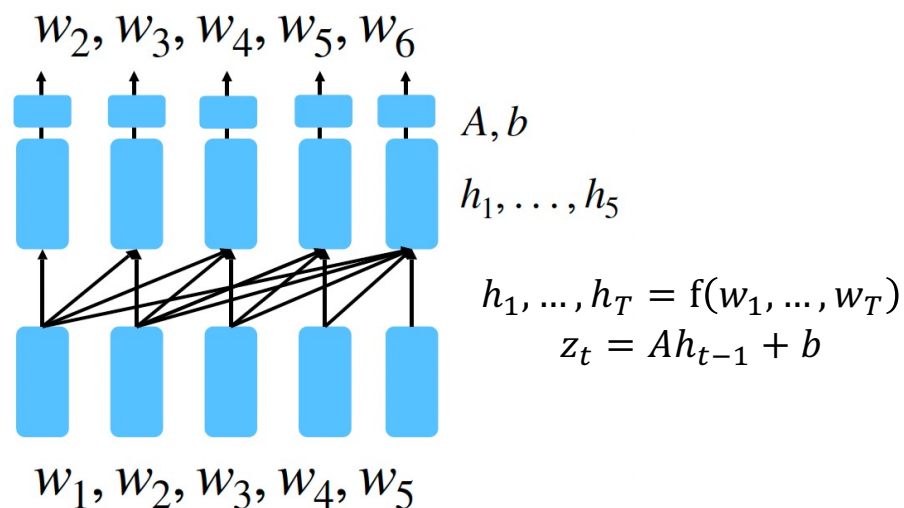
- Base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
- Large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- The larger the better!

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
Human (expert) [†]	-	85.0
Human (5 annotations) [†]	-	88.0

Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). 2019: 4171-4186.

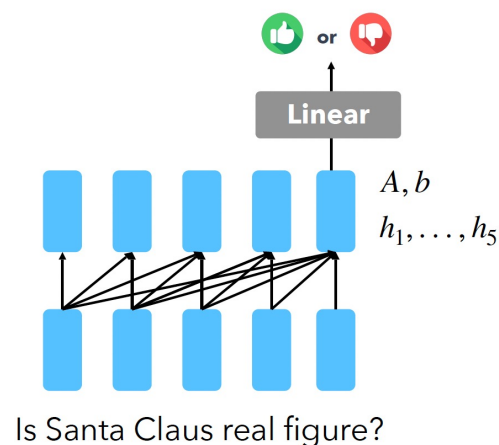
Autoregressive language modeling

Pre-trained model



$p_\theta(w_t | w_1, \dots, w_{t-1}, x)$: conditioned on a source context to generate from left-to-right

Pre-trained model for downstream tasks



Add a linear layer on top of the last hidden layer to make it a classifier!

Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training[J]. 2018.

| Autoregressive language modeling

$$\min_{\theta} \sum_{x \in D_{\text{train}}} \sum_t -\log p_{\theta}(w_t | w_{<t})$$

- θ : model parameters
- D_{train} : training data
- $p_{\theta}(w_t | w_{<t})$: conditional language modeling, conditioned on a source context to generate from left to right.

• Pros:

- The pre-training objective is unified and simple, and the training procedure is consistent with the inference process.
- Strong scaling behavior

• Cons:

- Only uses left-to-right context
- Autoregressive generation is slow
- Error accumulation during generation

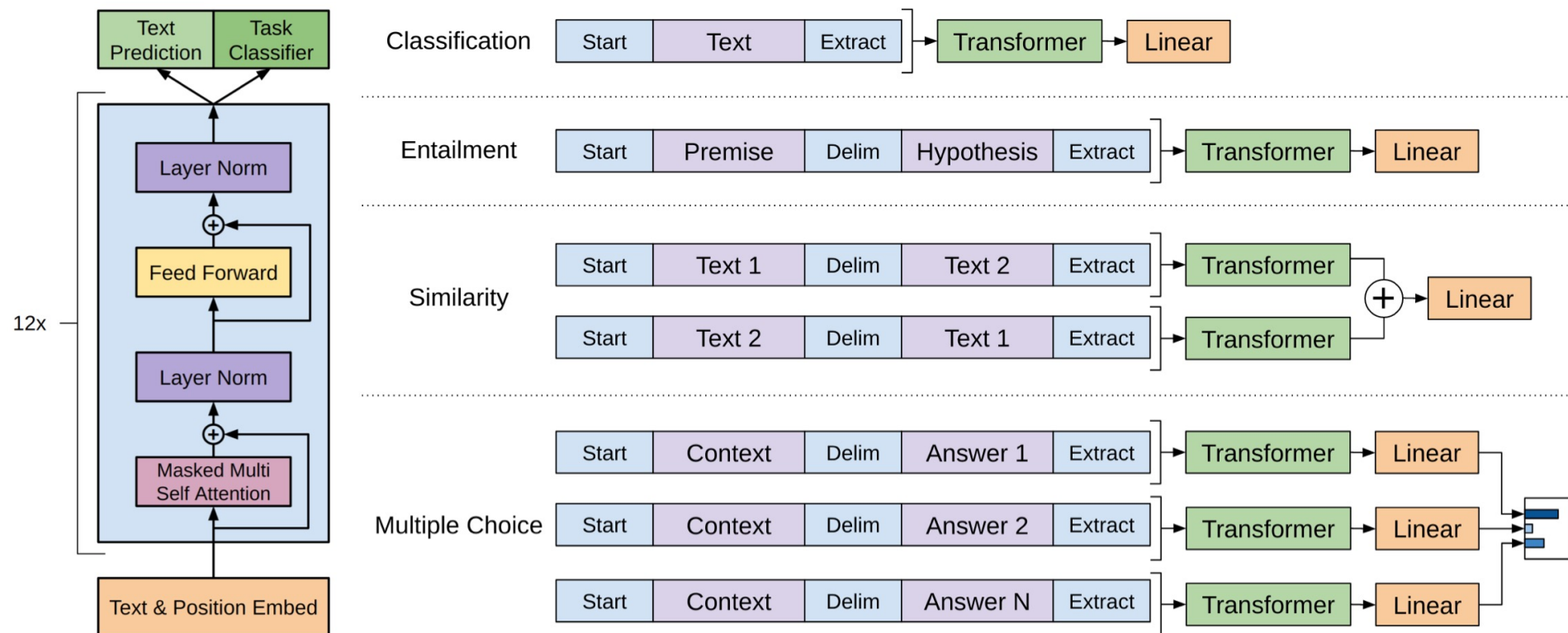
Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training[J]. 2018.

| Autoregressive language modeling: GPT

- GPT=Generative Pretrained Transformer
- GPT was a big success in 2018.
 - Transformer decoder with 12 layers, 117M parameters.
 - 768 dimensional hidden states, 3072-dimensional feed-forward hidden layers
 - Byte-pair encoding with 40,000 merges
 - Trained on BooksCorpus: over 7000 unique books.

Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training[J]. 2018.

Autoregressive language modeling: GPT



Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training[J]. 2018.

Autoregressive language modeling: GPT

- GPT results on various natural language inference datasets

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	<u>89.3</u>	-	-	-
CAFE [58] (5x)	80.2	79.0	<u>89.3</u>	-	-	-
Stochastic Answer Network [35] (3x)	<u>80.6</u>	<u>80.1</u>	-	-	-	-
CAFE [58]	78.7	77.9	88.5	<u>83.3</u>		
GenSen [64]	71.4	71.3	-	-	<u>82.3</u>	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	<u>82.1</u>	61.7
Finetuned Transformer LM (ours)	82.1	81.4	89.9	88.3	88.1	56.0

- GPT results on question answering datasets

Method	Story Cloze	RACE-m	RACE-h	RACE
val-LS-skip [55]	76.5	-	-	-
Hidden Coherence Model [7]	<u>77.6</u>	-	-	-
Dynamic Fusion Net [67] (9x)	-	55.6	49.4	51.2
BiAttention MRU [59] (9x)	-	<u>60.2</u>	<u>50.3</u>	<u>53.3</u>
Finetuned Transformer LM (ours)	86.5	62.9	57.4	59.0

Radford A, Narasimhan K, Salimans T, et al. Improving language understanding by generative pre-training[J]. 2018.

| Autoregressive language modeling: GPT

- **Example : Llama**

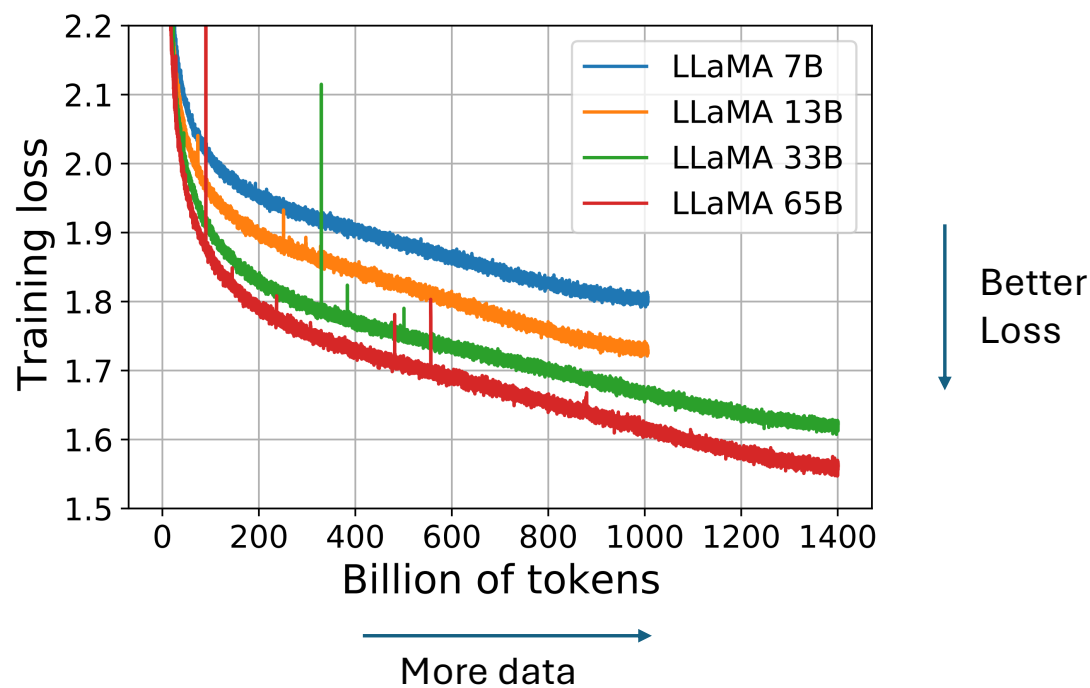
- Model: Transformer, {6.7B, 13B, 32B, 65B}
- Data: 1.4 trillion tokens, sources

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

Touvron H, Lavril T, Izacard G, et al. Llama: Open and efficient foundation language models[J]. arXiv preprint arXiv:2302.13971, 2023.

Autoregressive language modeling: GPT

- Llama: training loss



Touvron H, Lavril T, Izacard G, et al. Llama: Open and efficient foundation language models[J]. arXiv preprint arXiv:2302.13971, 2023.

Extra readings

- For a very good visualization of transformer, see the Youtube video
 - <https://www.youtube.com/watch?v=9-Jl0dxWQs8>
- For mechanistic interpretability of LLM, see the blog
 - <https://www.lesswrong.com>
- For a very good implementation from scratches, see the following:
 - <https://github.com/karpathy/nanoGPT> (by Andrej Karpathy)
 - <https://www.youtube.com/watch?v=kCc8FmEb1nY> (explaining the above codes).

| Demon

- Nothing is better than Karpathy's nanoGPT, just go there and read the codes and run them on your own computer for a pre-training.

Conclusion

- LLM does not go through the training-test paradigm, but uses the pre-training, mid-training, post-training paradigm.
 - Pre-training makes later stages easier.
 - Pre-training is the key to model transfer.
- Several factors contribute to LLM's success: data, architecture, objective, and hyperparameters.
- Two sorts of training objectives based on Transformer.
- Data becomes very important once the others are (almost) fixed.

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (short answer): write down the objective function of pre-training using auto-regression.
 - Q2 (T/F): Pretrained models can be further fine-tuned for other tasks.
 - Q3 (T/F): both BERT and GPT use transformer architecture, but are pre-trained using different training objective functions.
 - A1: $\sum_{x \in D_{\text{train}}} \sum_t -\log p_{\theta}(w_t | w_{<t})$
 - A2: T
 - A3: T